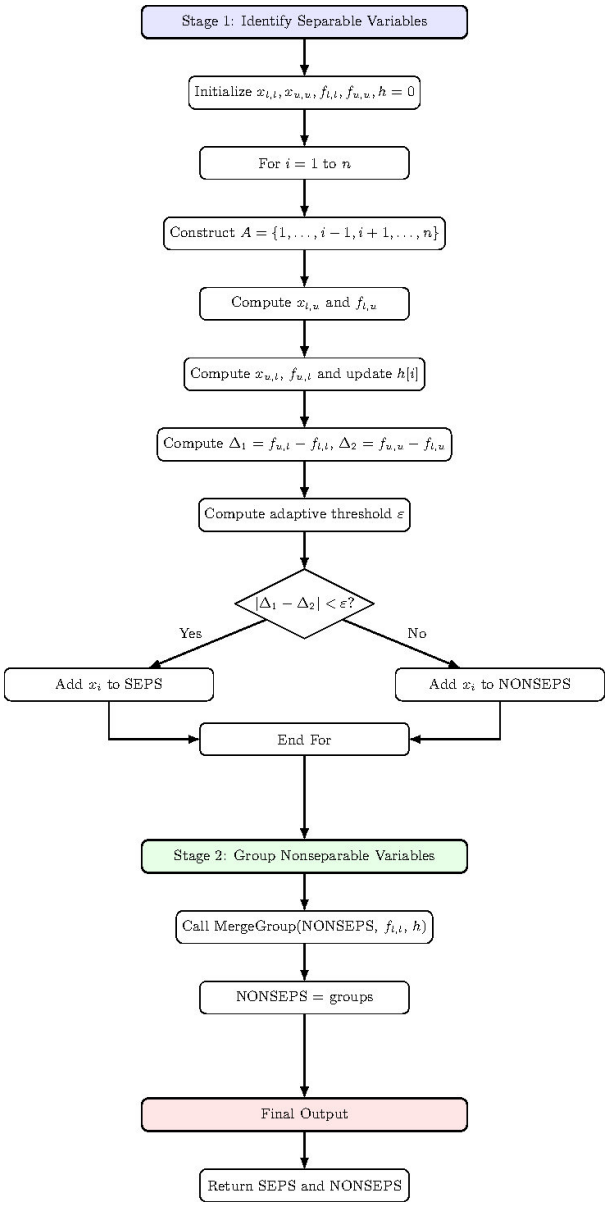
MDG(Merged Differential Grouping)

As an optimized variable grouping method, MDG is equiped with the following key features:

- **1. High effiency:** The computational complexity of MDG is $O\left(max\left{n, n_{ns}\times\log_2 k\right}\right)$, which is lower than the best complexity $O\left(n\log_2n\right)$ among FII, RDG, RDG2, and ERDG.
- **2. High grouping accuracy:** Firstly, MDG detects the subset-subset interaction ,rather than traditional variable-variable interction, which helps focusing on overall variables' interaction. Secondly, MDG employs adaptive threshold in order to better evaluate the interction of each variable sebsets. Last but not least, MDG adopts a binary-tree-based merging approach and calculations in each layer are independent, which avoids misjudgements caused by the accumulation of noises such as floating-point errors.
- **3. Parameter-free**

The grouping process in MDG can be divided into two stages. In stage 1, MDG identifies each variable as either seperable ot nonseperable variable. In stage 2, the nonseperable variables are assigned into different groups, based on the subset-subset interaction and a binary-tree-based iterative merging method.

The main procedure of MDG is as follows:



Algorithm 1 Procedure of MDG

Require: n: number of decision variables, c: a scale factor,
lb, **ub**: lower and upper bounds of the search space, respectively.
Ensure: **seps**: all separable variables,
nonseps: groups of all non-separable variables.

Initialize $\mathbf{x}_{l,l} = \mathbf{lb}$, $\mathbf{x}_{u,u} = \mathbf{ub}$, $f_{l,l} = f(\mathbf{x}_{l,l})$, $f_{u,u} = f(\mathbf{x}_{u,u})$,
$\mathbf{h} = \mathbf{0}$, $\mathbf{seps} = \varnothing$, $\mathbf{nonseps} = \varnothing$,
/*$\mathbf{h}$ records the historical perturbed value of each variable*/
For $i = 1$ to n /*identification of separable variables*/
  $A = \{1, \ldots, i-1, i+1, \ldots, n\}$.
  $\mathbf{x}_{l,u} = \mathbf{x}_{l,l}$, $\mathbf{x}_{l,u}[A] = \mathbf{ub}[A]$, $f_{l,u} = f(\mathbf{x}_{l,u})$.
  $\mathbf{x}_{u,l} = \mathbf{x}_{l,l}$, $\mathbf{x}_{u,l}[i] = \mathbf{ub}[i]$, $\mathbf{h}[i] = f_{u,l} = f(\mathbf{x}_{u,l})$.
  $\Delta_1 = f_{u,l} - f_{l,l}$, $\Delta_2 = f_{u,u} - f_{l,u}$.
  Calculate ϵ based on Eq. (15)
  If $|\Delta_1 - \Delta_2| < \epsilon$
    $\mathbf{seps} = \mathbf{seps} \cup \{x_i\}$.
  Else
    $\mathbf{nonseps} = \mathbf{nonseps} \cup \{x_i\}$.
  End If
End For
If $\mathbf{nonseps} \neq \varnothing$ /*grouping non-separable variables*/
  $(\mathbf{groups}, f_{nons})$ = MergeGroup($\mathbf{nonseps}, f_{l,l}, \mathbf{h}$). //Alg. 2
  $\mathbf{nonseps} = \mathbf{groups}$.
End If

The main improvement ideas of MDG

Binary-Tree-Based Iterative Merging

Binary-Tree-Based Iterative Merging is a divide-and-conquer grouping strategy used in MDG to efficiently spare nonseparable variables. Its core idea is to put variables into a binary tree and merge them level by level, detecting interactions only between subsets in the same level. This hierarchical merging design brings several key advantages:

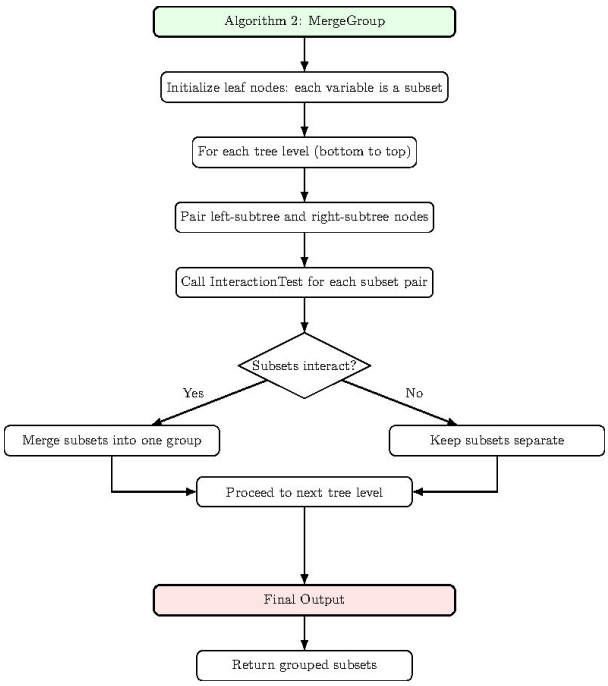
1. **Dramatically reduced computational complexity**

- Instead of checking all variable pairs ($O(n^2)$, like DG), the binary-tree structure only needs $O(n log_2n)$ interaction checks.
- Each level halves the number of nodes, so the consumption of computing resources is decreasing.

2. **Capture both direct and indirect interactions**

- When two interacting subsets merge at a lower level, the merged subset propagates upward. This allows MDG to automatically detect indirect multi-variable interactions, which classical DG often could not capture.

Binary-Tree-Based Iterative Merging works like this：

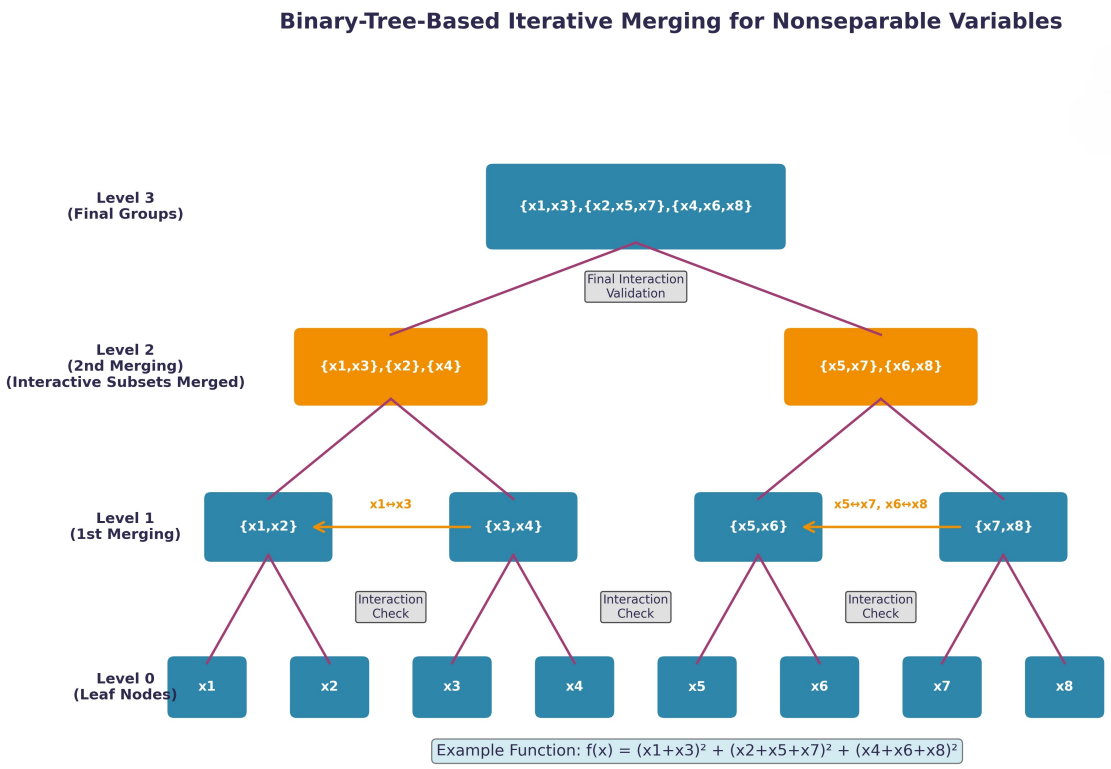


Algorithm 2 Procedure of Binary-Tree-Based Iterative Merging

Require: $dims$: The indexes of all nonseparable variables.
$f_{l,l}$: a historical unperturbed value. c: a scale factor.
$\mathbf{h}$: the historical perturbed value of each variable.
Ensure: $groups$: the interaction group of all non-separable variables.
f_{dims}: a historical perturbed value.

```
If |dims| == 1
    groups = dims, f_dims = h(dims).
    Return
End if
Divide dims into two equally-sized and mutually exclusive
    subset Ldims and Rdims.
(Lgroups, f_{u,l}) = MergeGroup(Ldims, f_{l,l}, h).
(Rgroups, f_{l,u}) = MergeGroup(Rdims, f_{l,l}, h.)
n_l = length(Lgroups), n_r = length(Rgroups).
x_{u,u} = lb, x_{u,u}[dims] = ub[dims], f_{u,u} = f_dims = f(x_{u,u}).
Δ_1 = f_{u,l} − f_{l,l}, Δ_2 = f_{u,u} − f_{l,u}.
Calculate ϵ based on Eq. (15).
If |Δ_1 − Δ_2| > ϵ
  If n_l == 1 ∧ n_r == 1
    groups = {Lgroups ∪ Rgroups}.
    Return
  End if
  (Lgroups_in, pTree)=BiSearch(Rgroups, Lgroups, f_{l,l},
                                f_{u,l}, f_{l,u}, f_{u,u}, null).
  If Lgroups_in ≠ ∅
    For i = 1 to |Lgroups_in|
      Lgi = Lgroups_in{i}.
      x_{u,l} = lb, x_{u,l}[Lgi] = ub[Lgi], f_{u,l} = f(x_{u,l}).
      x_{u,u} = x_{u,l}, x_{u,u}[Rdims] = ub[Rdims], f_{u,u} = f(x_{l,l})
      pTree.data = f_{l,u}, pTree → left = null,
      pTree → right = null.
      (Rgroups_in{i}, pTree)=BiSearch(Lgroups_in{i},
                                Rgroups, f_{l,l}, f_{u,l}, f_{l,u}, f_{u,u}, pTree).
    End for
  End if
  Merge the interacted subsets between Lgroups_in and
    Rgroups_in as Mgroups.
  Lgroups = Lgroups\Lgroups_in.
  Rgroups = (Rgroups\Rgroups_in) ∪ Mgroups.
End if
groups = Lgroups ∪ Rgroups.
```

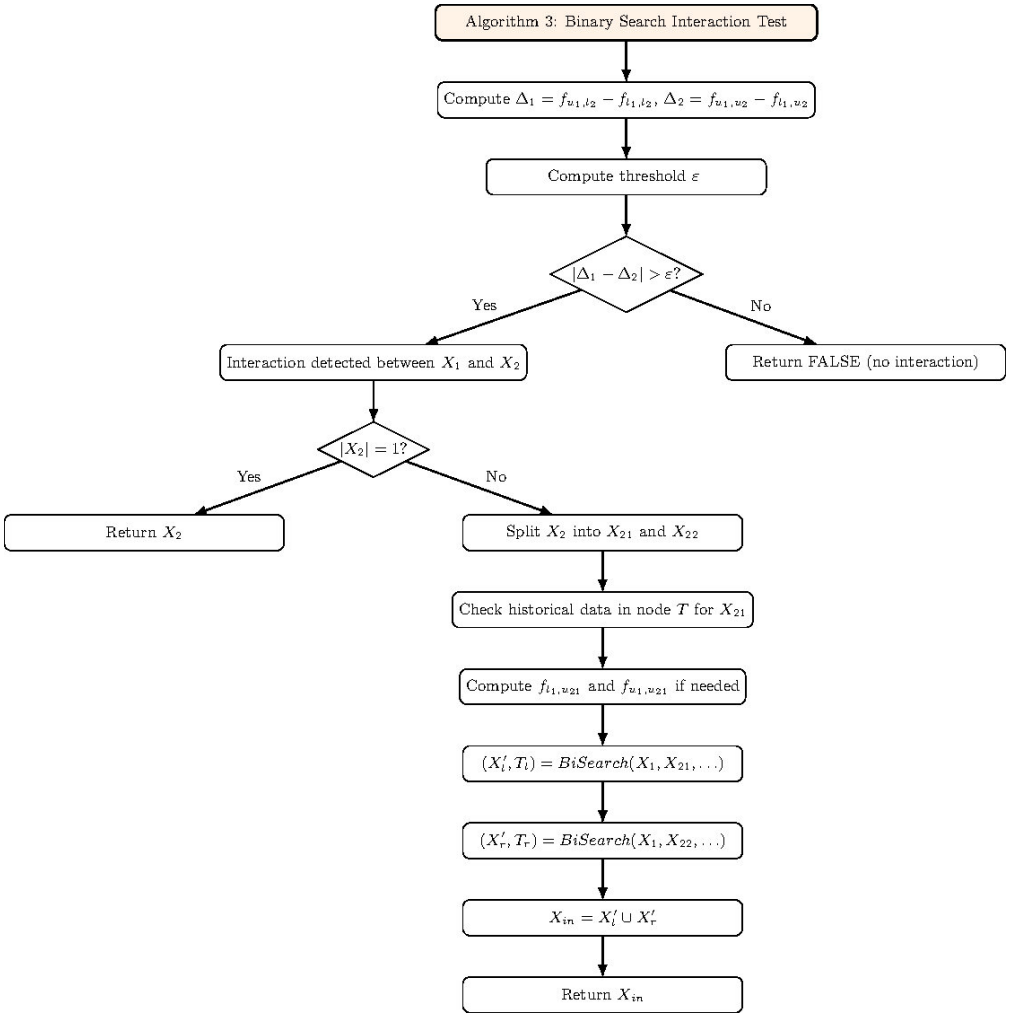
We could also take a simple function as example:



Binary Search

Binary search serves as a core efficient mechanism to detect subset–subset variable interactions in MDG. It recursively splits a candidate variable subset into two equal halves, checks for interactions between the target subset and each half using MDG's adaptive threshold, and reuses historical fitness evaluation information to minimize computational cost. This process narrows down interactive sub-subsets exponentially, enabling accurate and efficient variable grouping while reducing the number of fitness evaluations per interaction check, thus enhancing MDG's overall decomposition efficiency for LSGO.

The procedure of Binary Search is as follows:

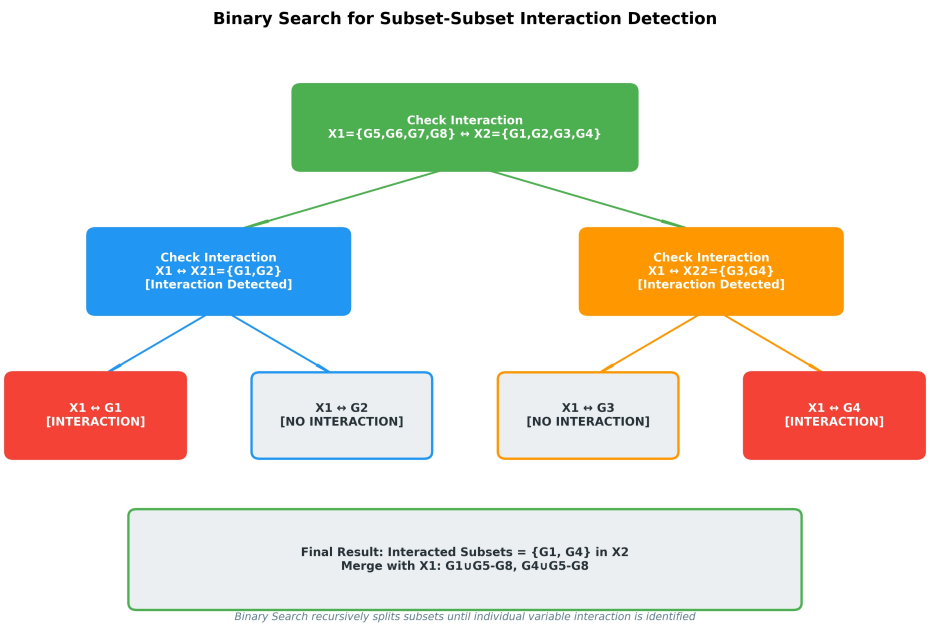


Algorithm 3 Procedure of Binary Search

Require: X_1, X_2: Two variable subsets.
$f_{l_1,l_2}, f_{u_1,l_2}, f_{l_1,u_2}, f_{u_1,u_2}$: Historical perturbed fitness values.
$\mathbf{x}_b$: the pre-perturbed decision vector for subset-subset interactions.
T: Current node in the binary search tree.
Ensure: X_{in}: the member subset of X_2 that interacts with X_1.

$\Delta_1 = f_{u_1,l_2} - f_{l_1,l_2}$, $\Delta_2 = f_{u_1,u_2} - f_{l_1,u_2}$
Calculate ϵ based on Eq. (15)
If $|\Delta_1 - \Delta_2| > \epsilon$ $\quad //X_1 \leftrightarrow X_2$
  If X_2 contains one member only
    $X_{in} = X_2$
  Else
    Divide X_2 into two parts X_{21} and X_{22} with an equal size
    If $T \rightarrow left \neq null$
      $f_{l_1,u_{21}} = T \rightarrow left.data$
    Else
      $\mathbf{x}_{l_1,u_{21}} = \mathbf{x}_b, \mathbf{x}_{l_1,u_{21}}[X_{21}] = \mathbf{ub}[X_{21}], f_{l_1,u_{21}} = f(\mathbf{x}_{l_1,u_{21}})$
      $T \rightarrow left.data = f_{l_1,u_{21}}$, $T \rightarrow right.data = f_{l_1,u_2}$
    End if
    $\mathbf{x}_{u_1,u_{21}} = \mathbf{x}_{l_1,u_{21}}, \mathbf{x}_{u_1,u_{21}}[X_1] = \mathbf{ub}[X_1]$,
    $f_{u_1,u_{21}} = f(\mathbf{x}_{u_1,u_{21}})$
    (X^l_{in}, lT)=BiSearch$(X_1, X_{21}, f_{l_1,l_2}, f_{u_1,l_2}, f_{l_1,u_{21}}, f_{u_1,u_{21}}, \mathbf{x}_b, T \rightarrow left)$
    (X^r_{in}, rT)=BiSearch$(X_1, X_{22}, f_{l_1,u_{21}}, f_{u_1,u_{21}}, f_{l_1,u_2}, f_{u_1,u_2}, \mathbf{x}_{l_1,u_{21}}, \mathbf{x}_b, T \rightarrow right)$
    $X_{in} = X^l_{in} \cup X^r_{in}$
    $T \rightarrow left = lT, T \rightarrow right = rT$
  End if
End if

Here is an example of Binary Search to check the interaction between X_1 and X_2



Historical evaluation information reuse

MDG stores historical perturbed values when a variable or a variable set is first perturbed. When MDG is checking new interactions that need to evaluate the same perturbation values, it reuses the stored values instead of calculation again. This helps save a lot of computing resources.

Adaptive threshold

MDG introduces an adaptive threshold based on multiple fitness values, the number of variables and the rounding error of floating numbers, in order to better evaluate the interaction of each pair of varible subsets.

The calculation method of ϵ is as follows: $$\epsilon = c \times 2^{-52} \times F_{max} \times dim$$

Where:

$F_{\max} = \max{f_1, f_2, f_3, f_4}$

$c = 0.003$ (a prespecified parameter)

2^{-52} (Floating-point error)

dim is the problem dimensionality.